The EAOS methodology defines a structured lifecycle for
Designing, deploying, and governing enterprise-grade AI systems

Project: Standard Framework Project

[Ref] 문서 · Context · Rule 연관 관계 맵

[산출물 ID: Ref-1 Doc-Content-Rule.docx]

항목	내용
프로젝트	Standard Framework Project
기준 Phase	Phase 1 — Vision & Role 정의 (완료)
원천 문서	1.1 Problem Framing / 1.2 Persona / 1.3 R&R Matrix / 1.4 Authority Model / 1.5 HITL Structure / 1.6 Governance Charter + Phase 0 SaaS Foundation
Context JSON	13개 (.cursor/context/*.json)
Rule MDC	8개 (.cursor/rules/*.mdc)
작성일	2026-03-09

1. 전체 구조 한눈에 보기

원천 문서 → Context JSON 변환 → Rule MDC 적용의 3계층 흐름

원천 문서 (Source Document)

(사람이 볼 수 있는 문서)

- 파일 형식 (md, word, pdf) 로 만들어서 보관
- Contenxt 파일로 변환을 요청할 때는 md 파일을 기준으로 요청 하면 됨
- 원천 문서 만들 때 AI 방법론 절차 준수
- 원천 문서 만들 때 AI Agent 에게 요청 하여 검토 하는 순서로

.cursor/context/

(AI Agent 가 효율적이고 일관되게 원천문서의 내용을 전달하기 위해 JSON 형태로 만들어 단계)

- (원천 문서를 Agent(claude) 에게 context 문서 json 으로 만들어 달라고 하면, 해당 문서를 json 파일로 만들어 줌 만들어 준 파일을 프로젝트 파일위치 에 가져다 놓으면 됨.
- Json 만들어 달라고 할 때 , 방법론의 단계 단위로 요청 하는 것이 좋음 (세부 문서가 나올 때 마다 요청을 하면, 이전에 만들어진 json 파일을 업데이하는경우가 발생 함.

.cursor/rules/

(AI Agent 가 .cusor/context 혹은 원천 문서를 분석하여 규정을 만들어 주는 단계)

- 원천 문서 혹은 context 내용을 AI Agent 에게 .제공 해주고 관련된 Rule 을 만들라고 요청
- Rule 만들어 달라고 할 때 , 방법론의 단계 단위로 요청 하는 것이 좋음 (세부 문서가 나올 때 마다 요청을 하면, 이전에 만들어진 Rule 파일을 업데이하는경우가 발생 함.
- 원천문서의 내용이 변경되면 context / rule 도 검토 하여 변경하여야 함

2. 원천 문서 → Context JSON 매핑

1.1 Problem Framing 하나에서 project_context / scope_context / governance_rules_context 3개의 JSON이 파생된다.

원천 문서	생성된 Context JSON	포함 핵심 내용
1.1 Problem Framing	project_context.json	문제 정의, 이해관계자, KPI, 가치 제안
1.1 Problem Framing (Scope 섹션)	scope_context.json	IS-01~09, OS-01~09 Scope 경계 정의
1.1 Problem Framing (Governance Rules 섹션)	governance_rules_context.json	AI 리스크 임계값, Zero-Trust, Approval 매트릭스
1.2 Persona Definition	persona_context.json	Persona 3종(P1~P3), Usage Scenario
1.3 Agent R&R Matrix	rr_matrix_context.json	RACI 매핑, Agent별 allowed/forbidden
1.4 Authority Model	governance.json (v1.1 업데이트)	Authority Level L0~L3, Agent별 레벨 결정
1.5 HITL Structure	hitl_context.json	P1/P2/P3 패턴, DB 스키마, SLA
1.6 Governance Charter	governance_charter.json	5대 원칙, 데이터 분류, 금지 행동
Phase 0 SaaS (0_0~0_C)	operation_standards.json	5대 원칙 요약, 운영 표준 통합
Phase 0 SaaS Foundation	saas_foundation.json	기술 스택, 공통 모듈, 아키텍처 결정
Phase 0 SaaS (Implementation)	saas_implementation.json	파일 생성 순서, 의존성 기준
Phase 2 Workflow Design	workflow_context.json	Agent 계층, Decision Gate, Escalation
Phase 3 Build	phase3_context.json	Boilerplate 연계 기준 (depends: saas_foundation + workflow + hitl)

3. 원천 문서 → Rule MDC 매핑

원천 문서	직접 기반 Rule	적용 방식
1.1 Problem Framing (Scope)	07_scope_governance.mdc	In-Scope/Out-Scope 코드 생성 경계, 갈등 해결 기준, MVP 품질 게이트
1.3 Agent R&R Matrix	00_master.mdc (참조)	Agent 코드 생성 시 역할 범위 확인 기준
1.4 Authority Model	00_master.mdc (Authority Level 섹션)	L0~L3별 코드 적용 기준표 직접 포함
1.5 HITL Structure	00_master.mdc (참조)	Phase 3 ApprovalGate 구현 기준 문서로 연결
1.6 Governance Charter (5대 원칙)	01_principles.mdc	5대 원칙 코드 적용 규칙 (최상위 우선순위)
1.6 Governance Charter (보안/데이터)	02_security.mdc	데이터 분류, RBAC, LLM 보안, API 키 규칙
Phase 0 (Context 관리 기준 0_2)	03_context_management.mdc	Context 3계층, 변경 절차, 버전 관리
Phase 0 SaaS Foundation	04_saas_stack.mdc	파일 생성 순서, DB/Auth/캐시/Agent 코드 규칙
Phase 0 (Quick Reference)	05_naming_errors.mdc	네이밍 규칙, 에러 코드 체계
Phase 0 (Quick Reference)	06_api_patterns.mdc	REST 패턴, React Query, 응답 포맷

4. Context JSON 간 의존 관계

파일 수정 시 이 의존 관계를 따라 연쇄 업데이트 여부를 확인할 것

```
project_context.json
    ├── scope_context.json                  (project_context 참조)
    └── operation_standards.json            (project_context 참조)
        └── persona_context.json            (op_standards + governance + project_context 참조)

governance.json
    ├── rr_matrix_context.json              (governance 참조)
    ├── workflow_context.json               (governance 참조)
    └── governance_rules_context.json
        ├── governance.json                 (Authority Level)
        ├── governance_charter.json         (5원칙, 데이터 분류)
        └── hitl_context.json               (P1/P2/P3 패턴)

saas_foundation.json
    └── saas_implementation.json            (saas_foundation 참조)

phase3_context.json  ← 복합 의존 (가장 마지막에 생성)
    ├── saas_foundation.json
    ├── workflow_context.json
    └── hitl_context.json
```

✧ Context Naming Rule 생성 규칙 고민 필요 (원천 문서와의 연계성 확보 필요)

5. Rule MDC 우선순위 및 참조 관계

```
00_master.mdc  ← 인덱스 (모든 context + 모든 rule 참조)
 │
 ├── [1순위] 01_principles.mdc     ← 5대 원칙 (최상위)
 │                                      ↑ 04_saas_stack.mdc 가 참조
 │                                      ↑ 07_scope_governance.mdc 가 참조
 │
 ├── [2순위] 02_security.mdc       ← 보안/데이터 규칙
 │                                      ↑ 04_saas_stack.mdc 가 참조
 │                                      ↑ 07_scope_governance.mdc 가 참조
 │
 ├── [3순위] 03_context_management.mdc    ← Context 구조 (독립)
 │
 ├── [4순위] 07_scope_governance.mdc      ← Scope 경계 + MVP 게이트
 │                 └── 참조: 01_principles, 02_security, 04_saas_stack
 │
 ├── [5순위] 00_master.mdc (본 파일)
 │
 ├── [6순위] 04_saas_stack.mdc            ← 기술 스택 규칙
 │                 └── 참조: 01_principles, 02_security
 │
 ├── [7순위] 05_naming_errors.mdc         ← 네이밍 규칙 (독립)
 │
 └── [8순위] 06_api_patterns.mdc          ← API 패턴 (독립)
```

6. 코드 생성 요청 시 Cursor 참조 흐름

```
Cursor 코드 생성 요청 수신
          │
          ▼
  00_master.mdc 로드  →  모든 context JSON 로드
          │
          ├── In-Scope 판단?        → 07_scope_governance.mdc
          │                              scope_context.json (IS/OS 목록)
          │
          ├── Agent 코드?           → rr_matrix_context.json (allowed/forbidden)
          │                              governance.json (Authority Level)
          │                              hitl_context.json (승인 게이트 위치)
          │
          ├── 보안 처리?            → 02_security.mdc
          │                              governance_charter.json (데이터 분류)
          │                              governance_rules_context.json (Zero-Trust)
          │
          ├── DB / API 코드?        → 04_saas_stack.mdc
          │                              saas_foundation.json (기술 결정)
          │                              saas_implementation.json (파일 순서)
          │
          ├── 네이밍?               → 05_naming_errors.mdc (독립 규칙)
          │
          ├── API 응답 패턴?        → 06_api_patterns.mdc
          │                              saas_foundation.json
          │
          └── 원칙 위반 여부?       → 01_principles.mdc (최우선 체크)
                                         operation_standards.json (5대 원칙 요약)
```

7. 문서별 영향 범위 요약

문서 수정 시 영향받는 Context / Rule 범위를 이 표로 먼저 확인한다.

문서	영향 Context	영향 Rule	영향 범위 요약
1.1 Problem Framing	3개 (project, scope, gov_rules)	1개 (07)	프로젝트 목표+Scope 경계+리스크 임계값
1.2 Persona	1개 (persona)	0개	UX 설계 컨텍스트
1.3 R&R Matrix	1개 (rr_matrix)	간접 (00 참조)	Agent별 실행 허용 범위
1.4 Authority Model	1개 (governance 일부)	직접 (00 포함)	모든 Agent 코드의 승인 게이트 여부 결정
1.5 HITL Structure	1개 (hitl)	간접 (00 참조)	Phase 3 ApprovalGate 구현의 직접 기준
1.6 Governance Charter	1개 (gov_charter)	2개 (01, 02)	모든 코드에 적용되는 5대 원칙 + 보안 규칙
Phase 0 (SaaS)	3개 (saas_f, saas_i, op_std)	4개 (03, 04, 05, 06)	기술 스택 표준 전체
Phase 2 (Workflow)	1개 (workflow)	0개	Agent 실행 흐름 + Gate 정의

8. 미완성 / Phase 이후 추가 예정 항목

⚠ *agents/ 서브폴더는 Phase 3 시작 직전에 generate_agent_contexts() 스크립트로 Global Context 기반 자동 생성.*

항목	현재 상태	생성 시점	비고
.cursor/context/agents/*.json (5개)	미생성	Phase 3 시작 시	Global Context에서 자동 생성
0_9_Release_Gate.json	미생성	Phase 4~5	Release Gate 기준
0_B_Exception_Approval.json	미생성	Phase 4~5	예외 승인 절차
0_5_KPI_Monitoring.json	미생성	Phase 5	KPI/모니터링 기준
0_6_Evolution_Loop.json	미생성	Phase 6	Evolution Loop 기준